Causal Diagnosis of Reinforcement Learning Performance in Vision-Based Multi-Robot Task Allocation with a Digital Twin

1st Yan Santos Leite
School of Electrical, Mechanical and Computer Engineering
Universidade Federal de Goiás (UFG)
Goiânia, Brazil
santosleiteyan@gmail.com

2nd Alisson Assis Cardoso
School of Electrical, Mechanical and Computer Engineering
Universidade Federal de Goiás (UFG)
Goiânia, Brazil
alsnac@ufg.br

***Abstract*—Multi-robot task allocation (MRTA) is commonly solved with greedy heuristics that are fast but cannot anticipate future demand. We train a reinforcement learning (RL) agent (Proximal Policy Optimization, PPO) for this task, with observations grounded in a camera-based digital twin: an overhead camera and a YOLOv8n detector localize a fleet of TurtleBot3 robots with a mean tracking error of 4.7 cm (mAP at 0.5 IoU of 0.995), replacing raw odometry as the position source for the allocation policy. We compare PPO against the greedy *nearest-free* heuristic using paired statistical tests over 1000 episodes. Contrary to our hypothesis, PPO did not outperform nearest-free: mean response time was about 12% worse, a statistically significant difference. PPO assigns tasks to busy robots in 4.2% of decisions, versus 0.3% for nearest-free, suggesting the training penalty was insufficient to teach this constraint. We tested this explanation with *action masking*, which prevents the policy from choosing busy robots by construction: the invalid-assignment rate dropped to 0%, yet response time remained about 11% worse than nearest-free. The constraint violation is therefore not the dominant cause of the gap, which we attribute instead to the small action space ($N = 3$) and training budget. We report this negative result together with the experiment that refuted our own explanation for it, since a hypothesis tested and falsified is more informative than an untested positive claim.**

***Index Terms*—multi-robot task allocation, deep reinforcement learning, proximal policy optimization, action masking, digital twin, YOLOv8, ROS 2, reproducibility, negative results**

I. Introduction

Coordinating a fleet of mobile robots to service spatially and temporally distributed demands (multi-robot task allocation, MRTA) is a core problem in multi-robot coordination [1], with applications spanning service robotics to warehouse automation [2], [3]. Greedy heuristics such as nearest-free assignment are simple, fast, and respect operational constraints (e.g., never assigning a busy robot) by construction. Their problem is that they only look at the current demand: they assign the closest idle robot without considering that this same robot might be better used, more efficiently, for a demand that arrives shortly after. Reinforcement learning (RL) is a natural candidate to correct this myopia, since a learned policy can, in principle, trade a locally suboptimal assignment for a globally better outcome.

There is a second, largely independent challenge. To decide which robot to send, the allocator needs to know where each robot currently is, and obtaining that in real time is neither trivial nor free. Wheel odometry, the simplest method, estimates position from how much the wheels have turned, and accumulates error over time. More precise methods, such as SLAM, correct for that drift but are expensive to deploy on real hardware. We use a low-cost alternative: a ceiling-mounted camera observing the operating area, with a YOLOv8n detector that identifies each robot in the image. We call this system a *digital twin*: it publishes each robot's position in real time, without accumulating error over time.

We hypothesized that a learned policy with a two-step demand lookahead would outperform nearest-free by trading locally suboptimal assignments for globally better outcomes. This paper reports that the hypothesis is false in our setting, and identifies why. Our contributions are:

- A camera-based digital twin (4.7 cm mean localization error, mAP@0.5 = 0.995) that feeds the observations of an RL allocator, with the full pipeline validated on three TurtleBot3 Waffle robots in Gazebo.
- A negative result established with paired statistical tests: PPO does not outperform the nearest-free heuristic, and the digital-twin noise is a second-order factor in that outcome.
- A causal experiment that refutes our own leading explanation for the gap. PPO assigns tasks to busy robots 13 times more often than the heuristic, so we removed busy robots from the action set entirely via action masking. The violation rate fell to zero and the performance gap remained, redirecting the diagnosis toward the action space and training budget.

II. Related Work

Classical MRTA relies on market-based auctions [4], greedy assignment [1], and centralized optimization, which offer strong constraint guarantees but limited foresight. Reproducibility and evaluation practices remain open problems in deep RL [6]: variance across random seeds alone can produce

statistically significant differences, motivating significance testing rather than comparing average returns alone, a gap we address with paired tests. Orr and Dutta [7] survey multi-agent RL applications in multi-robot systems, identifying scalability and the lack of standardized simulation benchmarks as open challenges.

Learning-based allocation has been explored across several formulations, including multi-agent CTDE settings [5] and dual-agent self-play frameworks [3] that jointly learn task selection and robot assignment. In warehouse domains, recent work includes attention-based centralized policies [13] and decentralized two-level MDP approaches that couple task allocation with reactive navigation [2].

A second line of work targets scalability and generalization. Paul et al. [14] use capsule attention networks to generalize across problem sizes without retraining, and Dai et al. [15] apply an attention encoder-decoder with a leader-follower training scheme to dynamic coalition formation, validated on physical drones and matching an exact MIP solver at two orders of magnitude lower runtime. Fenoy et al. [18] report that an attention-based allocator for capacitated pickup-and-delivery outperformed the state of the art, but also that one naively engineered input feature degraded it below a random baseline. Street et al. [16] instead model *stochastic* task announcement with continuous-time Markov chains, a probabilistic (non-RL) alternative that reduces waiting time by 74.6% in simulation.

Closest in setup to ours, Elfakharany and Ismail [17] train an end-to-end decentralized PPO policy mapping raw sensor input to steering commands on simulated TurtleBot3 robots, allocating tasks implicitly rather than through an explicit decision layer as we do. What separates our work from all of the above is the question we ask: rather than proposing a new architecture and reporting where it wins, we take a standard PPO formulation that *loses* to a greedy heuristic and run a controlled experiment to identify why. On perception, YOLO-family detectors [8] are common for vision-based tracking [9], but prior work evaluates detection quality in isolation, without ablating its effect on a learned allocation policy while holding all else fixed, the ablation we run here. Negative-result reporting remains rare in robotics and RL; we follow Henderson et al.'s call for paired statistical evaluation [6]. Our causal diagnosis also draws on Huang and Ontañón [10], who show that action masking produces a valid policy gradient and outperforms soft penalization of invalid actions. Unlike that RTS setting, where masking closes the performance gap, in our domain it eliminates the structural violation without closing the gap, evidence that the underlying cause differs.

III. Digital Twin and RL Formulation

The digital twin consists of a statically mounted overhead camera, a YOLOv8n detector fine-tuned on 375 manually annotated frames (one to three TurtleBot3 Waffle robots; example detections in Fig. 1), and a coordinate-projection module mapping bounding-box centers to world-frame (x, y) positions, published on ROS2 topic `/robot_detections`. The detector reaches mAP@0.5 = 0.995 with 36 ms/frame inference on CPU. A static 8-pose benchmark gives 1.24 cm mean error; over a continuous multi-robot tracking sequence (2031 samples, live Nav2 navigation) the mean error is 4.7 cm, with 100% of detections within 15 cm of ground truth (Fig. 2). The roughly $4\times$ increase from static to dynamic is consistent with motion blur and detection-to-pose latency, not a detector limitation. This dynamic error is the precision level used in all downstream experiments.

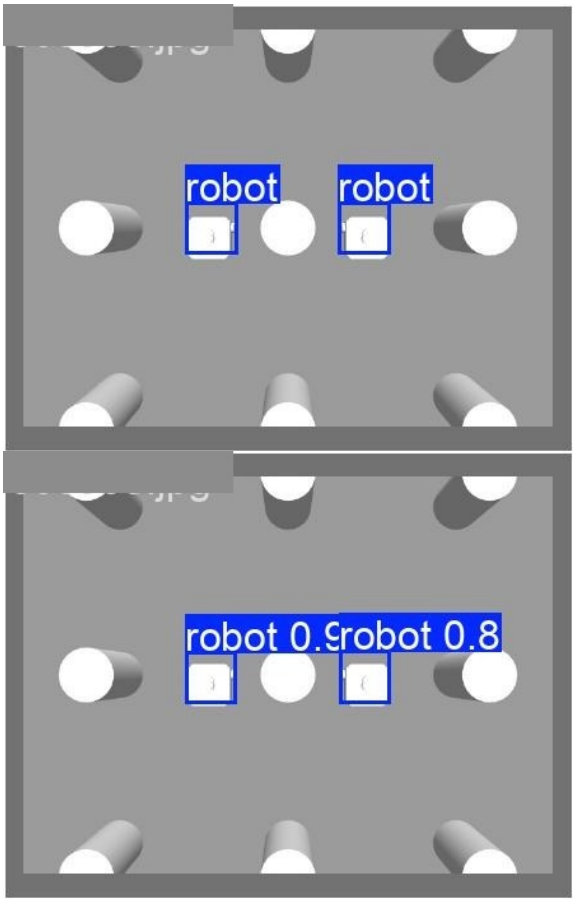

Fig. 1. Two of the fleet's TurtleBot3 Waffle robots as seen by the overhead camera: ground-truth annotation (top) vs. detector prediction (bottom) in a scene from the validation set. Full three-robot experiments are quantified in Sections IV–V.

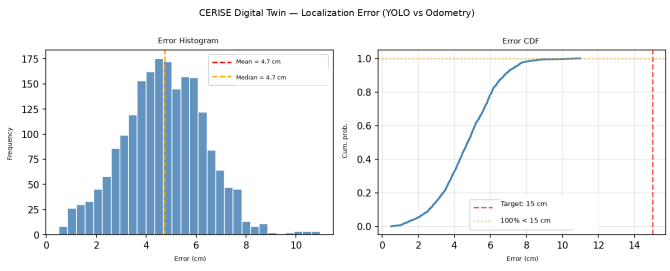

Fig. 2. Digital twin localization error over a continuous multi-robot tracking sequence. Mean error is 4.7 cm; 100% of detections fall within 15 cm of ground truth.

We formulate task allocation as a Markov Decision Process (MDP): at each new demand arrival, a single centralized agent decides, based on the current state of the system, which of $N = 3$ robots services it. The 21-dimensional state describes everything the agent observes at that instant: each robot's (x, y) position (from the digital twin or odometry, depending on the experimental arm), how long until each robot becomes free, the origin and destination of the demand that just arrived, and the origin and destination of the next two demands already visible in the queue, a lookahead signal that lets the agent weigh the future impact of each choice, not just the immediate demand. The action is the discrete choice of which of these N robots should service the current demand. The reward is the negative response time $r = -(t_{\text{wait}} + t_{\text{travel}})$: the faster a demand is serviced, from arrival until the chosen robot starts executing it, the higher the reward. This corrects an earlier formulation that penalized only travel time to the demand, ignoring how long the chosen robot would still take to become free; under that formulation, a busy-but-nearby robot could look as good as an idle-but-distant one, which under-weights queueing and biases the policy toward idle robots even when they are far away. We train with PPO [11] via Stable-Baselines3 [12] for 500,000 timesteps ($\sim$2 h on CPU), at library default hyperparameters (Table I) to isolate the effect of observation source and masking rather than PPO tuning; Gazebo is reserved for final-stage physical validation, not training. Both PPO variants learn (Fig. 3): mean episodic reward improves from ~ -21 to ~ -17.7 over 500k steps, but neither crosses the nearest-free reference (-16.7); PPO(odom) converges slightly faster, confirming the 4.7 cm digital-twin noise is a second-order effect.

TABLE I
TRAINING HYPERPARAMETERS (IDENTICAL ACROSS ALL VARIANTS)

Hyperparameter	**Value**
Policy / network	MlpPolicy, π=[64,64], V=[64,64]
Learning rate	3×10^{-4}
n_{steps} (per env) / batch	2048 / 64
γ / GAE λ / clip range	0.99 / 0.95 / 0.2
Parallel envs / seed	8 / 42

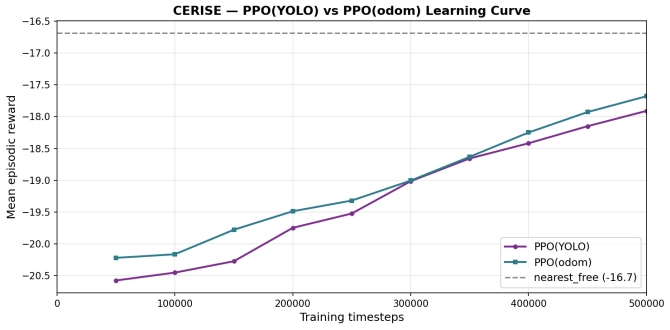

Fig. 3. PPO learning curves for the YOLO and odometry observation variants, with the nearest-free baseline as a reference line.

IV. Experimental Setup

We compare PPO against three baselines: *random* and *round-robin*, which serve as a performance floor with no notion of proximity or queueing, and *nearest-free*, which assigns the closest idle robot and is infeasible by construction when none is idle. To isolate the digital-twin effect, we train PPO(YOLO) and PPO(odom), identical except for the position source used in the observation (4.7 cm Gaussian noise for YOLO vs. accumulated odometry drift for odom); using matched seeds across both variants ensures the only difference between them is that noise source, not training luck. Each comparison evaluates 1000 paired episodes, using the same demand sequence and seed for both policies, which lets us report not only whether the difference is statistically significant (Wilcoxon signed-rank) but also its effect size (Cliff's delta) and uncertainty (bootstrap confidence intervals). To check whether these conclusions hold under different demand pressures, a load sweep covers six inter-arrival regimes from 30 s to 10 s (500 episodes/point). The full pipeline, from camera to YOLO, through the allocator and Nav2, to the TurtleBot3 Waffle robots, was validated end-to-end on three robots in Gazebo Classic (ROS2 Humble); the RL policy itself, however, trains only in the analytical environment. To causally test whether the soft constraint of never assigning a busy robot drives PPO's poor performance, we additionally train PPO(YOLO, masked): identical to PPO(YOLO) except the action distribution is restricted every step to currently idle robots, via *action masking* [10] using sb3-contrib's MaskablePPO, with the same hyperparameters and training budget. We omit a PPO(odom, masked) variant since the digital-twin noise effect already proved second-order (Section V), so testing it would not change the experiment's qualitative conclusion.

V. Results

Table II summarizes moderate-load performance. Nearest-free outperforms both PPO variants on the three time-based metrics (mean response time, p95, and per-episode response cost), by about 12% on mean response time. On the fourth metric PPO is *better*: both variants achieve lower load imbalance (0.135) than nearest-free (0.149), indicating the policy did learn to distribute work more evenly across the fleet, but that this gain did not translate into faster service. The difference between PPO(YOLO) and PPO(odom) is small compared to how far either falls behind nearest-free, confirming the digital-twin noise is a second-order effect. The Wilcoxon test confirms this advantage is statistically significant in both comparisons ($p < 0.001$, large Cliff's δ). The p95-to-mean ratio is comparable across policies ($\approx$1.47–1.56, Fig. 4), and is in fact slightly lower for PPO than for nearest-free. PPO's disadvantage therefore comes from a shift of the entire response-time distribution rather than from a tail of catastrophic episodes.

Table III and Fig. 5 show that PPO's disadvantage relative to nearest-free narrows as load increases, from +19.0% at low load to +7.7% at high load, though nearest-free remains faster throughout the tested range.

TABLE II

POLICY COMPARISON AT MODERATE LOAD (INTER-ARRIVAL 12S, 1000 PAIRED EPISODES). RESPONSE TIME IS WAIT + TRAVEL; RESPONSE COST IS THE PER-EPISODE SUM OF RESPONSE TIMES. LOWER IS BETTER FOR ALL METRICS.

Policy	Mean resp.	p95 resp.	Resp. cost	Imbal.
Nearest-free	**17.86 s**	**27.77 s**	**357.18 s**	0.149
PPO (YOLO)	20.01 s	29.46 s	400.20 s	**0.135**
PPO (odom)	19.83 s	29.37 s	396.69 s	**0.135**

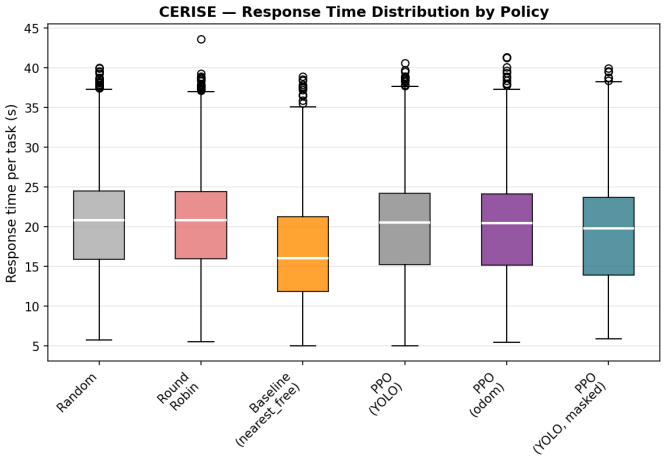

Fig. 4. Response-time distribution across evaluated policies, including the masked PPO variant.

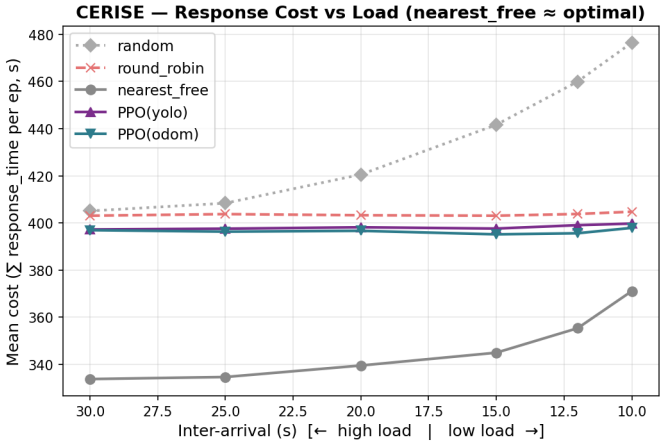

Fig. 5. Response time as a function of demand load (inter-arrival time) for nearest-free and PPO(YOLO).

TABLE III

RESPONSE TIME BY POLICY ACROSS LOAD REGIMES

Inter-arrival	Nearest-free	PPO(YOLO)	Difference
30 s (low)	333.8 s	397.3 s	+19.0%
12 s (mod.)	357.2 s	400.2 s	+12.0%
10 s (high)	371.1 s	399.8 s	+7.7%

VI. DIAGNOSING THE NEGATIVE RESULT

PPO learns, and even gets a two-step demand lookahead the greedy baseline lacks, yet still fails to beat nearest-free. We investigated three explanations. Soft vs. hard constraint enforcement: nearest-free never assigns a busy robot, since that constraint is built into the algorithm itself; PPO can only learn to avoid this indirectly through the reward penalty, and still gets it wrong in 4.2% of decisions versus 0.3% for nearest-free, the largest single contributor we found. Limited action space: with only $N = 3$ robots, each decision offers at most three alternatives, and few of them are meaningfully different in outcome, which caps any anticipatory advantage a lookahead could exploit. Training budget: 500k steps produced clear learning but not convergence, though PPO's own response cost stays nearly flat across load regimes (397–400 s, Table III) while nearest-free's grows substantially under high load, suggesting the plateau is not purely a budget artifact.

Causal test. To test directly whether that constraint violation was the cause, we trained PPO(YOLO, masked) and re-evaluated it with unmasked PPO(YOLO) and nearest-free over a fresh 1000-episode run at moderate load, so all three policies were compared under identical conditions. Masking achieves its structural goal: the invalid-assignment rate drops from 4.2% to 0.0%. Yet mean response time barely improves, from about 12% worse than nearest-free without masking to about 11% worse with it, roughly a 1-point gain. Nearest-free still significantly outperforms the masked variant ($p < 0.001$, Cliff's $\delta = 0.636$), an effect size of the same order of magnitude as without masking ($\delta = 0.703$).

The causal hypothesis is refuted. The explanation that looked most plausible before the test does not hold up: eliminating the violation by construction barely closes the gap. Weight shifts back to the small action space and training budget, and a further hypothesis emerges: by preventing the policy from even considering busy robots, masking also removes any learning signal about the consequence of that choice, which may reduce exploration diversity without offering an anticipation advantage in return.

Limitations. Our claim is bounded by four choices. First, we use library-default PPO hyperparameters, so our result concerns PPO as commonly applied rather than the ceiling of RL for this problem; a tuned agent may well close the gap. Second, $N = 3$ robots is a small action space, and our own diagnosis points to it as a dominant factor, so these findings should not be extrapolated to large fleets. Third, the policy trains in an analytical environment and is validated end-to-end in Gazebo, but not on physical robots, so sim-to-real transfer remains untested. Fourth, 500k timesteps produced learning without convergence. Each of these is a direct target for the future work below.

VII. CONCLUSION AND FUTURE WORK

We integrated a validated camera-based digital twin with a PPO-based multi-robot task allocator and found that the learned policy does not beat a greedy nearest-free heuristic. Action masking removed the most visible symptom, an

invalid-assignment rate 13 times higher than the baseline, without recovering the performance gap, which points to the action space and training budget instead.

The practical reading is that at this scale a greedy heuristic is already close to what a learned policy can deliver, while costing nothing to train. Our load sweep suggests where that balance may change: PPO's response cost stays flat as demand grows while nearest-free's rises, so the crossover point, if it exists, lies at higher load or larger fleets than we tested. Establishing whether it exists is the natural next step, and it requires the larger action space and longer training budget our diagnosis points to, alongside a policy architecture better suited to the problem structure than an MLP [13], [14], physical validation beyond simulation, and a Decentralized POMDP formulation under centralized training with decentralized execution [5].

ACKNOWLEDGMENT

The authors thank CERISE (Center of Excellence in Intelligent Wireless Networks and Advanced Services), hosted at the School of Electrical, Mechanical and Computer Engineering (EMC) of the Universidade Federal de Goiás (UFG) and supported by the Goiás State Research Foundation (FAPEG), for providing the infrastructure used in this work.

REFERENCES

[1] B. P. Gerkey and M. J. Matarić, "A formal analysis and taxonomy of task allocation in multi-robot systems," *Int. J. Robot. Res.*, vol. 23, no. 9, pp. 939–954, 2004.

[2] A. Agrawal, S. Hariharan, A. S. Bedi, and D. Manocha, "DC-MRTA: Decentralized multi-robot task allocation and navigation in complex environments," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2022, pp. 11711–11718.

[3] A. Pal, A. Chauhan, and M. Baranwal, "Together we rise: Optimizing real-time multi-robot task allocation using coordinated heterogeneous plays," in *Proc. Int. Conf. Auton. Agents Multiagent Syst. (AAMAS)*, 2025.

[4] M. B. Dias, R. Zlot, N. Kalra, and A. Stentz, "Market-based multirobot coordination: A survey and analysis," *Proc. IEEE*, vol. 94, no. 7, pp. 1257–1270, 2006.

[5] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Proc. NeurIPS*, 2017.

[6] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," in *Proc. AAAI*, 2018.

[7] J. Orr and A. Dutta, "Multi-agent deep reinforcement learning for multi-robot applications: A survey," *Sensors*, vol. 23, no. 7, p. 3625, 2023.

[8] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLO," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics. Accessed: Jul. 15, 2026.

[9] I. Ahmed, S. Din, G. Jeon, F. Piccialli, and G. Fortino, "Towards collaborative robotics in top view surveillance: A framework for multiple object tracking by detection using deep learning," *IEEE/CAA J. Autom. Sinica*, vol. 8, no. 7, pp. 1253–1270, 2021.

[10] S. Huang and S. Ontañón, "A closer look at invalid action masking in policy gradient algorithms," in *Proc. Int. FLAIRS Conf.*, vol. 35, 2022.

[11] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, arXiv:1707.06347.

[12] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable reinforcement learning implementations," *J. Mach. Learn. Res.*, vol. 22, no. 268, pp. 1–8, 2021.

[13] A. Agrawal, A. S. Bedi, and D. Manocha, "RTAW: An attention inspired reinforcement learning method for multi-robot task allocation in warehouse environments," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2023, pp. 1393–1399.

[14] S. Paul, P. Ghassemi, and S. Chowdhury, "Learning scalable policies over graphs for multi-robot task allocation using capsule attention networks," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2022, pp. 8815–8822.

[15] W. Dai, A. Bidwai, and G. Sartoretti, "Dynamic coalition formation and routing for multirobot task allocation via reinforcement learning," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024.

[16] C. Street, B. Lacerda, M. Mühlig, and N. Hawes, "Right place, right time: Proactive multi-robot task allocation under spatiotemporal uncertainty," *J. Artif. Intell. Res.*, vol. 79, pp. 137–171, 2024.

[17] A. Elfakharany and Z. H. Ismail, "End-to-end deep reinforcement learning for decentralized task allocation and navigation for a multi-robot system," *Appl. Sci.*, vol. 11, no. 7, p. 2895, 2021.

[18] A. Fenoy, J. Zagoli, F. Bistaffa, and A. Farinelli, "Attention for the allocation of tasks in multi-agent pickup and delivery," in *Proc. ACM/SIGAPP Symp. Appl. Comput. (SAC)*, 2024, pp. 622–629.